INDIAN DATA CLUB

INDIAN DATA CLUB

Day 20 (26/11) : Window Functions - Aggregate Window Functions

🎯 *Objective*

To understand and apply **Aggregate Window Functions** in SQL—specifically how to use functions like **SUM() OVER**, **AVG() OVER**, and window frames to calculate **running totals**, **moving averages**, and **partition-wise aggregates** without collapsing rows as GROUP BY does.

🔍 *Topics Covered*

- **SUM() OVER** → Calculate totals per partition or running totals.
- **AVG() OVER** → Calculate averages per partition or moving averages.
- **Running Totals** → Cumulative sum from start to current row.
- **Moving Averages** → Rolling average over a defined window (e.g., last 3 rows).

INDIAN DATA CLUB

SUM() OVER

Used for: Total per partition or running total

Syntax:

```
SUM(column) OVER (PARTITION BY col ORDER BY col2)
```

Example

Running total of sales by date.

```
SELECT
  date,
  sales,
  SUM(sales) OVER (ORDER BY date) AS running_total
FROM sales_table;
```

INDIAN DATA CLUB

AVG() OVER

Used for: Average per partition or moving average
Syntax:

```
AVG(column) OVER (PARTITION BY col ORDER BY col2 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
```

Example
3-day moving average

```
SELECT
  date,
  sales,
  AVG(sales) OVER (
      ORDER BY date
      ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moving_avg_3
FROM sales_table;
```

INDIAN DATA CLUB

Running Totals

Used for: Cumulative total from first row to current row
Syntax:

```
SUM(col) OVER (ORDER BY col2 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

Example

```
SELECT
  day,
  revenue,
  SUM(revenue) OVER (ORDER BY day) AS
running_total
FROM revenue_table;
```

INDIAN DATA CLUB

MOVING AVERAGES

Used for: Rolling average of last N rows
Syntax:

```
AVG(col) OVER (ORDER BY col2 ROWS BETWEEN N PRECEDING AND CURRENT ROW)
```

Example

```
SELECT
  date,
  aqi,
  AVG(aqi) OVER (
      ORDER BY date
      ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moving_avg_7
FROM air_quality;
```

INDIAN DATA CLUB

Practice Questions:

```sql
-- Calculate running total of patients admitted by week for each service.
SELECT
  service,
  week,
  patients_admitted,
  SUM(patients_admitted) OVER (PARTITION BY service ORDER BY week) AS running_patients
FROM services_weekly;
```

service	week	patients_admitted	running_patients
emergency	1	32	32
emergency	2	28	60
emergency	3	32	92
emergency	4	32	124
emergency	5	25	149
emergency	6	34	183
emergency	7	33	216
emergency	8	26	242
emergency	9	28	270
emergency	10	17	287
emergency	11	16	303
emergency	12	28	331
emergency	13	23	354
emergency	14	15	369
emergency	15	17	386
emergency	16	23	409
emergency	17	21	430
emergency	18	18	448
emergency	19	16	464
emergency	20	21	485
emergency	21	18	503

INDIAN DATA CLUB

Practice Questions:

```sql
-- Find the moving average of patient satisfaction over 4-week periods.
SELECT
  week,
  patient_satisfaction,
  ROUND(
      AVG(patient_satisfaction) OVER (
          ORDER BY week
          ROWS BETWEEN 3 PRECEDING AND CURRENT ROW
      ),
      1
  ) AS moving_avg_4
FROM services_weekly;
```

week	patient_satisfaction	moving_avg_4
1	67	67.0
1	83	75.0
1	97	82.3
1	84	82.8
2	75	84.8
2	96	88.0
2	73	82.0
2	79	80.8
3	73	80.3
3	63	72.0
3	95	77.5
3	82	78.3
4	83	80.8
4	74	83.5
4	67	76.5
4	64	72.0
5	93	74.5
5	81	76.3

INDIAN DATA CLUB

Practice Questions:

```sql
-- Show cumulative patient refusals by week across all services.
SELECT
  week,
  patients_refused,
  SUM(patients_refused) OVER (
      ORDER BY week
  ) AS cumulative_refusals
FROM services_weekly;
```

week	patients_refused	cumulative_refusals
1	44	302
1	85	302
1	164	302
1	9	302
2	141	583
2	0	583
2	140	583
2	0	583
3	145	789
3	39	789
3	21	789
3	1	789
4	125	1025
4	1	1025
4	109	1025
4	1	1025
5	363	1495
5	44	1495

INDIAN DATA CLUB

Daily Challenge:

```
-- Create a trend analysis showing for each service and week:
-- week number, patients_admitted, running total of patients admitted (cumulative),
-- 3-week moving average of patient satisfaction (current week and 2 prior weeks),
-- and the difference between current week admissions and the service average.
-- Filter for weeks 10-20 only.
```

```
SELECT
    service,
    week,
    patients_admitted,
    SUM(patients_admitted) OVER (
        PARTITION BY service
        ORDER BY week
    ) AS running_total_admitted,
    AVG(patient_satisfaction) OVER (
        PARTITION BY service
        ORDER BY week
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS moving_avg_3week,
    patients_admitted
        - AVG(patients_admitted) OVER (PARTITION BY service)
        AS diff_from_service_avg
FROM services_weekly
WHERE week BETWEEN 10 AND 20
ORDER BY service, week;
```

service	week	patients_admitted	running_total_admitted	moving_avg_3week	diff_from_service_avg
emergency	10	17	17	81.0000	-2.5455
emergency	11	16	33	89.5000	-3.5455
emergency	12	28	61	82.3333	8.4545
emergency	13	23	84	81.0000	3.4545
emergency	14	15	99	72.0000	-4.5455
emergency	15	17	116	80.6667	-2.5455
emergency	16	23	139	75.6667	3.4545
emergency	17	21	160	80.0000	1.4545
emergency	18	18	178	75.6667	-1.5455
emergency	19	16	194	88.0000	-3.5455
emergency	20	21	215	91.0000	1.4545
general_m...	10	35	35	95.0000	-2.8182
general_m...	11	35	70	85.0000	-2.8182
general_m...	12	33	103	80.0000	-4.8182
general_m...	13	33	136	72.6667	-4.8182
general_m...	14	47	183	76.3333	9.1818
general_m...	15	53	236	83.0000	15.1818
general_m...	16	39	275	87.0000	1.1818
general_m...	17	33	308	83.3333	-4.8182
general_m...	18	25	333	85.3333	-12.8182
general_m...	19	43	376	85.3333	5.1818